

BUT 3 Science des données, Niort

Projet Perceptron : Discrimination d'une image

Introduction aux réseaux de neurones – P. Carré – F. Equilbey

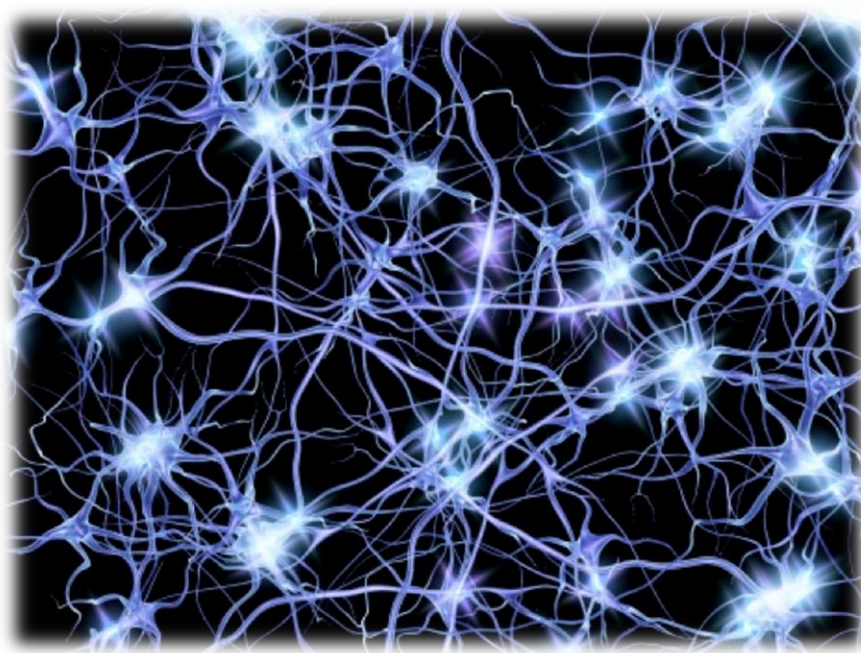


Table des matières

Introduction et contexte.....	2
Approche basée modèle.....	2
1. Méthodologie	2
1.1. Préparation des données (Data Engineering)	2
1.2. Architecture du modèle (MLP)	3
2. Analyse des résultats	4
2.1. Courbes d'apprentissage	4
2.2. Performance sur le jeu de test	5
2.3. Simulation du système de discrimination	6
2.4. Comparaison des hyperparamètres	6
3. Conclusion et perspectives (Comparaison Hyperparamètres).....	7
Approche « Deep-learning » (Basée Data)	8
1. Méthodologie	8
1.1 Préparation des données	8
1.2. Architectures testées.....	8
2. Analyse des résultats.....	9
2.1. Comparaison des Modèles	9
2.2. Performance détaillée (Meilleur Modèle).....	10
2.3. Tests Visuels.....	11
Conclusion générale du projet	12
Bilan comparatif	12
Contraintes et enseignements techniques.....	13
Perspectives d'amélioration	13

Introduction et contexte

L'objectif de ce projet est de réaliser un système de discrimination (classification) d'images réelles issues de la base de données Wang. Cette base contient 1000 images réparties équitablement en 10 classes sémantiques (Jungle, Plage, Monuments, Bus, Dinosaures, Éléphants, Fleurs, Chevaux, Montagne et Plats).

* * *

Approche basée modèle

Pour cette première approche, nous allons nous baser sur une stratégie "basée modèle" (Model-Based). Au lieu de travailler sur les pixels bruts, nous allons utiliser des descripteurs FCTH (Fuzzy Color and Texture Histogram) pré-calculés, disponibles dans le fichier WangSignatures.xls.

1. Méthodologie

1.1. Préparation des données (Data Engineering)

Les données reçues sont brutes. Nous avons alors mis en place une chaîne de traitement comprenant :

1. **Chargement** : Importation de la matrice **1000 x 192** (192 étant la taille du vecteur FCTH, de caractéristiques).
2. **Étiquetage** : Génération automatique du vecteur de labels (**y**) en exploitant la structure séquentielle du fichier (100 images par classe).
3. **Séparation du jeu de données** : Division du jeu de données en trois ensembles distincts pour garantir une évaluation non biaisée :
 - **Train (70%)** : Pour l'apprentissage des poids synaptiques.
 - **Validation (15%)** : Pour le réglage des hyperparamètres et l'arrêt précoce (*Early Stopping*).

- **Test (15%)** : Pour l'évaluation finale.
4. **Normalisation** : Application d'un `StandardScaler` (centrage-réduction) calculé uniquement sur le jeu d'entraînement, puis appliqué aux jeux de validation et de test. Cette étape est cruciale pour la convergence du MLP.

Extrait du code de préparation :

```
# Normalisation des descripteurs (StandardScaler)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# Encodage des labels en vecteurs binaires (One-Hot)
y_train_enc = to_categorical(y_train, num_classes=10)
```

1.2. Architecture du modèle (MLP)

Nous avons conçu un réseau de neurones de type **Perceptron Multi-Couches (MLP)** "Full-Connected" via le framework Keras/TensorFlow. La structure retenue est la suivante :

- **Couche d'entrée** : 192 neurones (correspondant aux descripteurs FCTH).
- **Couches cachées** :
 - Dense (256 neurones, activation ReLU) + BatchNormalization.
 - Dense (128 neurones, activation ReLU) + BatchNormalization.
- **Régularisation** : Utilisation de couches **Dropout** (taux de 0.3) après chaque couche cachée pour limiter le sur-apprentissage.
- **Couche de sortie** : 10 neurones avec activation **Softmax** pour obtenir une probabilité d'appartenance à chaque classe.

L'optimiseur utilisé est **Adam** avec un taux d'apprentissage initial de 0.001 et la fonction de coût est la **Categorical Crossentropy**.

Code de définition du modèle :

```

model = Sequential()
model.add(Dense(256, activation='relu', input_shape=(192,)))
model.add(BatchNormalization())
model.add(Dropout(0.3))
model.add(Dense(128, activation='relu'))
# ...
model.add(Dense(10, activation='softmax'))

```

2. Analyse des résultats

2.1. Courbes d'apprentissage

Le modèle a été entraîné sur 100 époques avec un mécanisme d'arrêt précoce (*Early Stopping*) paramétré sur la perte de validation (*val_loss*), qui a stoppé l'entraînement à l'époque 25 pour éviter la dégradation du modèle.

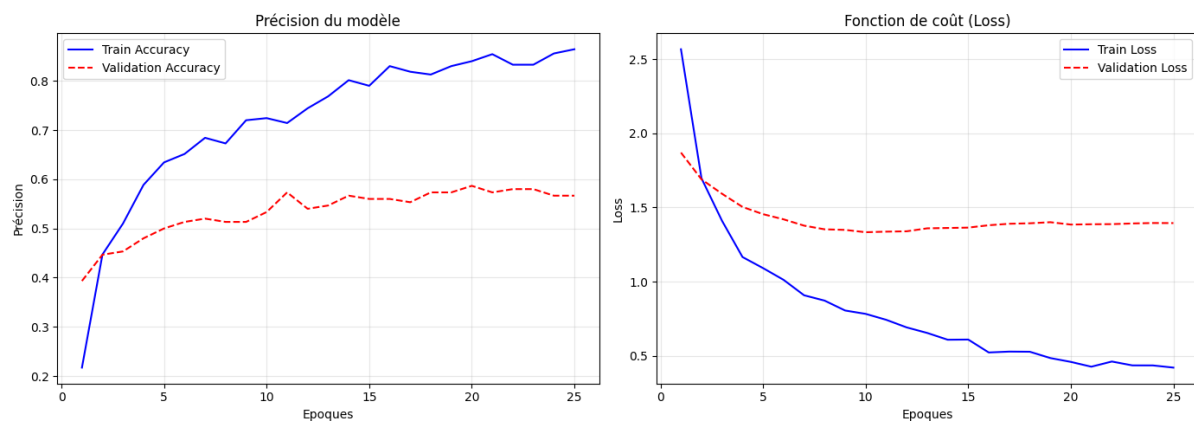


Figure 1: Courbes de Précision et de Perte

Analyse : On observe un phénomène classique de **sur-apprentissage (Overfitting)** :

- La précision sur l'entraînement (courbe bleue) continue de progresser jusqu'à atteindre ~85%.
- La précision sur la validation (courbe rouge) sature rapidement autour de 55-58%.
- Cela indique que le modèle a une capacité suffisante pour mémoriser les exemples d'entraînement, mais que les descripteurs FCTH ne contiennent peut-être pas assez d'information généralisable pour distinguer parfaitement les classes inédites.

2.2. Performance sur le jeu de test

Le modèle atteint une précision globale (**Accuracy**) de **47.33%** sur le jeu de test. Bien que supérieure au hasard (10%), cette performance montre les limites des descripteurs FCTH pour séparer ces classes.

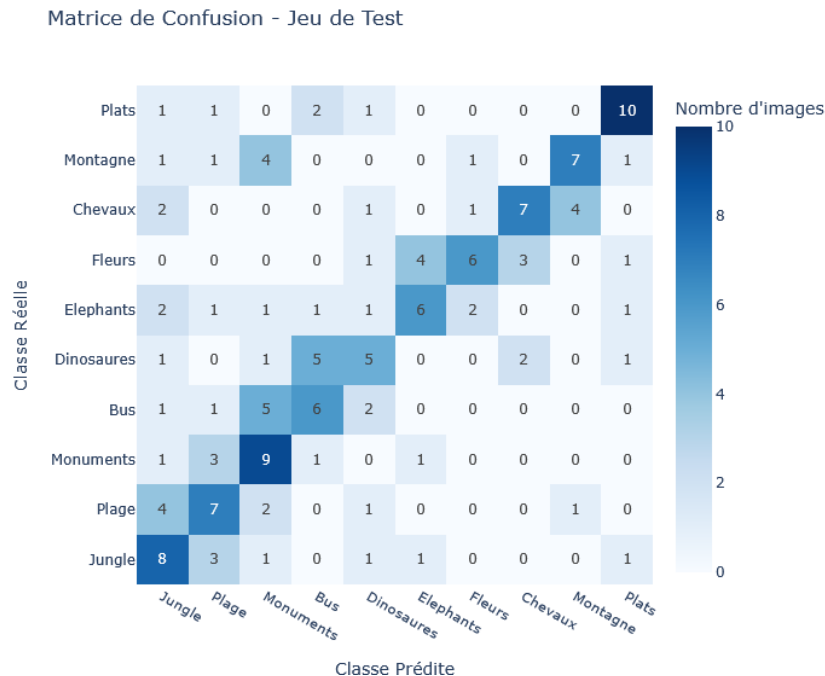


Figure 2 : Matrice de Confusion

Analyse détaillée par classe : En observant la matrice de confusion et le rapport de classification, nous notons de fortes disparités :

- **Meilleures performances** : Les classes **Plats** (F1-score 0.67) et **Jungle** sont bien reconnues. Les descripteurs de couleur/texture fonctionnent bien ici (ex: plats souvent colorés et centrés).
- **Confusions notables** :
 - **Chevaux vs Montagne** : 7 montagnes correctement prédites, mais 4 ont été classés comme "Chevaux". Cela s'explique par les fonds d'images (herbe/ciel) que le FCTH capture sans comprendre la forme de l'animal.
 - **Plage** : Cette classe agit comme un "attracteur" de faux positifs (confondue avec Jungle et Monuments), probablement à cause de la texture du sable ou du ciel bleu dominant.
 - **Bus vs Dinosaures** : Le modèle peine à distinguer ces classes, probablement car le FCTH capture mal les formes géométriques complexes, se concentrant trop sur les textures globales.

2.3. Simulation du système de discrimination

Conformément au cahier des charges, nous avons simulé l'utilisation réelle du système : un utilisateur soumet une image inconnue, et le système renvoie une classe et un taux de confiance.

Test sur 5 images aléatoires :

```
--- TEST DU SYSTÈME : PRÉSENTATION D'IMAGES INCONNUES ---  
Image #113 (Vrai: Elephants) -> Système: Elephants (82.9%) -> ✓ SUCCÈS  
Image #142 (Vrai: Monuments) -> Système: Monuments (80.9%) -> ✓ SUCCÈS  
Image #140 (Vrai: Montagne) -> Système: Fleurs (31.1%) -> ✗ ERREUR  
Image #91 (Vrai: Jungle) -> Système: Monuments (23.1%) -> ✗ ERREUR  
Image #45 (Vrai: Fleurs) -> Système: Plats (49.0%) -> ✗ ERREUR
```

Figure 3 : Renvoi du terminal pour les tests sur images aléatoires

- **Succès** : Image #113 (Éléphants) correctement prédite avec **82.9%** de confiance.
- **Succès** : Image #142 (Monuments) correctement prédite avec **80.9%** de confiance.
- **Erreur** : Image #140 (Montagne) incorrectement prédite comme "Fleurs" (confiance 31.1%).
- **Erreur** : Image #45 (Fleurs) incorrectement prédite comme "Plats" (confiance 49.0%).
- **Erreur** : Image #91 (Jungle) incorrectement prédite comme "Monuments".

Constat : Le système est capable de donner une réponse probabiliste. On remarque que lors des erreurs, la confiance est souvent plus faible (< 50%), ce qui pourrait permettre de filtrer les réponses incertaines dans une application réelle.

2.4. Comparaison des hyperparamètres

Nous avons testé 4 architectures différentes pour déterminer si la complexité du réseau améliorerait les résultats.

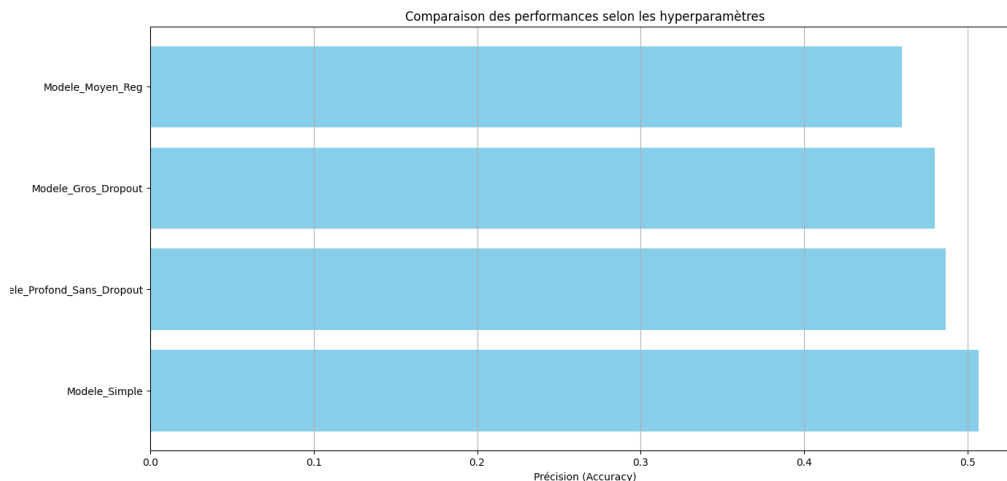


Figure 4 : Comparaison des performances

Configuration	Architecture (Neurones)	Dropout	Accuracy test
Modele_Simple	[64]	0.0	50.67%
Modele_Profond_Sans_Dropout	[256, 128, 64, 32]	0.0	48.67%
Modele_Gros_Dropout	[512, 256, 128]	0.5	48.00%
Modele_Moyen_Reg (Actuel)	[128, 64]	0.3	46.00%

Analyse critique : Contre toute attente, c'est le **Modèle Simple** (une seule couche cachée de 64 neurones) qui obtient la meilleure performance (50.67%). Cela s'explique par la taille réduite du jeu de données (700 images d'entraînement). Les modèles plus profonds ou complexes (comme le "Gros Dropout" ou le "Moyen") ont trop de paramètres à ajuster par rapport au peu d'information contenue dans les descripteurs FCTH, ce qui mène à une convergence difficile ou à un sur-apprentissage rapide malgré la régularisation.

3. Conclusion et perspectives (Comparaison Hyperparamètres)

L'approche par descripteurs FCTH ("Model-Based") permet de mettre en place une chaîne de traitement complète et fonctionnelle. Cependant, elle plafonne autour de **47-50% de précision**. Les erreurs sont dues à la perte d'information spatiale : le descripteur "résumé" l'image à des statistiques de couleurs et textures, empêchant la distinction fine entre des classes visuellement proches (ex: Cheval dans un pré vs Montagne verdoyante).

Pour la suite du projet, l'approche **Deep Learning (Data-Based)** utilisant des **CNN (Réseaux de Neurones Convolutifs)** sur les images brutes devrait surmonter cet obstacle en apprenant directement des filtres capables de détecter des formes complexes (pattes, têtes, roues de bus).

* * *

Approche « Deep-learning » (Basée Data)

Dans cette seconde partie, nous abandonnons les descripteurs pré-calculés pour adopter une approche "end-to-end" : nous fournissons les images brutes (pixels) au réseau, qui doit apprendre lui-même à extraire les caractéristiques pertinentes via des couches de convolution.

1. Méthodologie

1.1 Préparation des données

Contrairement à l'approche précédente, les données d'entrée sont ici des tenseurs de pixels.

- **Redimensionnement et contraintes matérielles** : Les images ont été redimensionnées en **128 x 128 pixels** (3 canaux RVB). Cette réduction par rapport au format standard (256 x 256) a été opérée afin de s'adapter aux **capacités mémoire (RAM) de la machine utilisée**, permettant ainsi l'entraînement des modèles sans surcharge système tout en préservant l'information visuelle essentielle.
- **Normalisation** : Les valeurs des pixels [0, 255] ont été normalisées entre [0, 1] pour faciliter la convergence de l'algorithme de descente de gradient.
- **Partitionnement** : Nous conservons la même répartition stricte : 700 images d'entraînement, 150 de validation et 150 de test.

1.2. Architectures testées

Nous avons implémenté et comparé deux architectures de **Réseaux de Neurones Convolutifs (CNN)** :

A. Modèle Simple (Suggéré) Une structure minimaliste imposée par le sujet pour tester l'extraction de base :

- 1 couche de Convolution (32 filtres, 3 x 3) + ReLU.
- 1 couche de MaxPooling (2 x 2) pour réduire la dimensionnalité.

- Aplatissement (Flatten) et couche de sortie Dense (Softmax).

B. Modèle Avancé (Optimisé) Une structure plus profonde inspirée de VGGNet, conçue pour extraire des caractéristiques hiérarchiques (formes simples -> formes complexes) :

- 3 blocs de Convolution (16, 32, 64 filtres) avec *Padding* pour conserver la taille spatiale.
- Utilisation de **BatchNormalization** pour stabiliser l'apprentissage.
- Ajout de **Dropout** (0.2 à 0.5) progressif pour lutter contre le sur-apprentissage.

2. Analyse des résultats

2.1. Comparaison des Modèles

Nous avons entraîné les deux modèles et comparé leur précision sur le jeu de test.

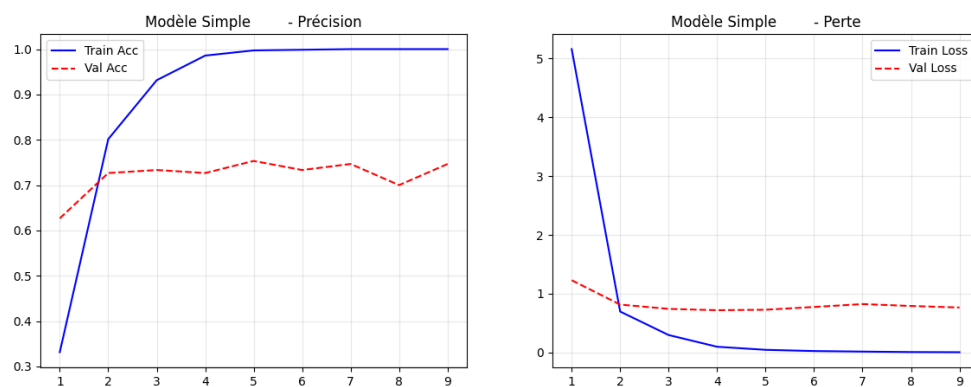


Figure 5 : Courbes d'apprentissage Modèle Simple

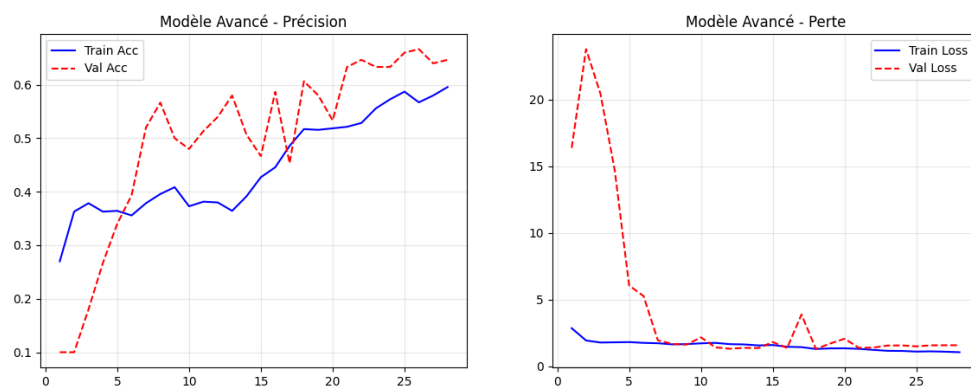


Figure 6 : Courbes d'apprentissage Modèle Avancé

Résultats :

- **Modèle Simple : 70.67%** de précision. Convergence très rapide (dès 5 époques) vers une solution stable.
- **Modèle Avancé : 65.33%** de précision. Les courbes montrent une instabilité et un sur-apprentissage plus marqué (la perte de validation remonte vite), signe que le modèle est trop complexe pour le faible volume de données (700 images).

C'est donc le **Modèle Simple** qui est retenu comme meilleur modèle.

2.2. Performance détaillée (Meilleur Modèle)

Avec une précision de **71%**, l'approche CNN surpasse largement l'approche par descripteurs FCTH (47%).

Matrice de Confusion CNN

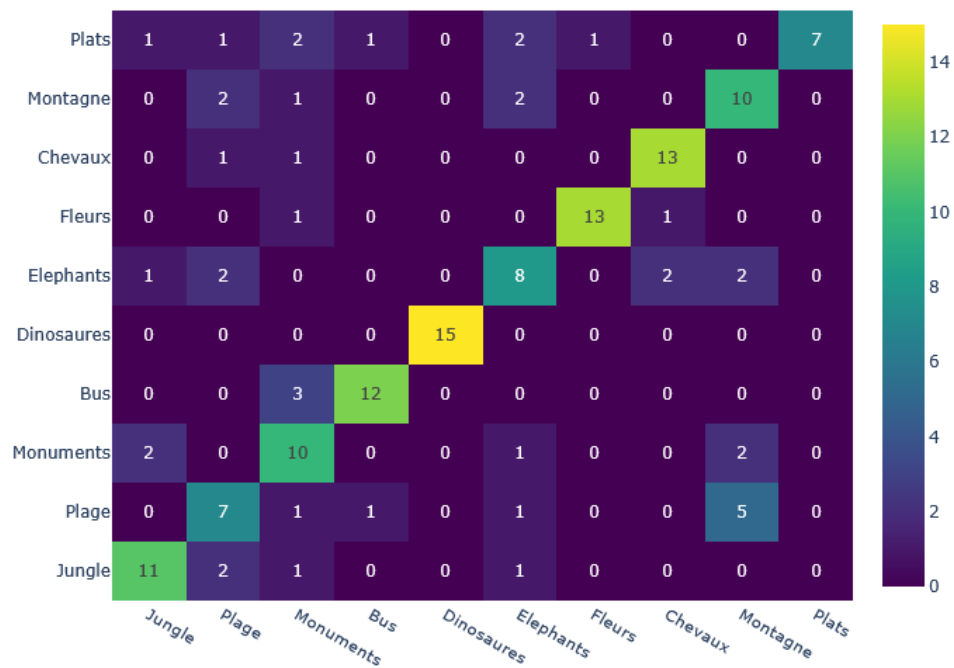


Figure 7 : Matrice de Confusion CNN

Analyse par classe :

- **Excellente distinction :**
 - **Dinosaures (F1-score 1.00)** : 100% de réussite. Le CNN détecte parfaitement les traits dessinés sur fond blanc, ce que le FCTH ne voyait pas.

- **Fleurs (F1-score 0.90)** : Très bien reconnues grâce aux formes et couleurs locales.
- **Bus (F1-score 0.83)** : Les contours rectangulaires sont bien captés par les convolutions.
- **Confusions persistantes :**
 - **Plage (F1-score 0.47)** : Encore confondue avec "Montagne" ou "Jungle". La distinction reste difficile même pour un CNN simple qui manque peut-être de contexte global.

2.3. Tests Visuels

Nous avons soumis des images inconnues au modèle pour valider son comportement.

Exemples de prédictions :

Pred: Bus (0.86)



Figure 8 : Prédiction de "bus", correcte

Pred: Montagne (0.87)



Figure 9 : Prédiction de "montagne", incorrecte

Pred: Plage (0.71)



Figure 10 : Prédiction de "plage", correcte

1. **Bus** : Correctement identifié avec **86%** de confiance. Le réseau a bien repéré la forme caractéristique.
2. **Plage (réelle) prédite Montagne (0.87)** : C'est une erreur intéressante car l'image contient une montagne en arrière-plan et de l'eau, donc Le réseau a focalisé sur la montagne ("partie saillante") plutôt que sur le sable ("contexte"), ce qui explique l'erreur avec une forte confiance.
3. **Plage** : Correctement identifiée (0.71).

* * *

Conclusion générale du projet

Ce projet nous a permis de comparer deux paradigmes de vision par ordinateur sur la base Wang :

Approche	Type	Précision max	Avantages	Inconvénients
Descripteurs (FCTH)	Feature Engineering	~47-50%	Rapide, explicable	Perd la structure spatiale
Deep Learning (CNN)	Representation Learning	~71%	Performance, apprend les formes	Nécessite plus de données/calculs

Bilan comparatif

Les résultats démontrent la supériorité incontestable de l'approche **Deep Learning**. Avec une précision de **71%**, le modèle CNN surpasse de **24 points** l'approche par descripteurs

(plafonnant à **47%**). Cet écart s'explique par la nature des traitements : là où le descripteur FCTH agrège des statistiques globales de couleurs et textures en perdant la structure de l'image (confondant par exemple un bus rouge et une fleur rouge), le CNN effectue un **apprentissage de représentation**. Il a su construire automatiquement des filtres hiérarchiques capables d'identifier des contours et des formes complexes, rendant la classification plus robuste.

Contraintes et enseignements techniques

La réussite du projet a néanmoins été conditionnée par deux facteurs limitants :

- **Les ressources matérielles** : La mémoire vive (RAM) disponible a imposé un redimensionnement des images en 128 x 128 pixels, réduisant la finesse des détails accessibles au modèle.
- **La faible volumétrie ("Small Data")** : Avec seulement 100 images par classe, nous avons observé un phénomène de sur-apprentissage rapide sur les architectures complexes. Paradoxalement, c'est notre **architecture CNN simple** qui a offert la meilleure capacité de généralisation, prouvant qu'un modèle plus léger est souvent préférable lorsque les données sont rares.

Perspectives d'amélioration

Pour franchir le palier des 80% de précision et résoudre les confusions sémantiques persistantes (notamment entre les classes "Plage" et "Montagne"), deux pistes d'amélioration semblent incontournables :

- L'intégration de l'**Augmentation de Données** (rotations, zooms) pour multiplier artificiellement les exemples d'entraînement.
- Le recours au **Transfer Learning** (via des réseaux comme VGG16 ou ResNet), qui permettrait de bénéficier d'extracteurs de caractéristiques pré-entraînés sur des millions d'images, palliant ainsi la taille réduite de notre jeu de données.

En conclusion, ce projet confirme que si les méthodes basées sur les descripteurs restent utiles pour leur rapidité et leur simplicité, l'approche Deep Learning est incontournable pour des tâches de classification complexe, à condition de maîtriser l'équilibre entre la complexité du modèle et la quantité de données disponibles.